

EXP NO: 04
DATE:

INTERACTIVE CHARTS WITH PLOTLY

Aim:

To learn the basics of data visualization using Matplotlib by creating simple line, bar, and scatter charts from a sample dataset, and to understand how to add titles, axis labels, legends, and grids.

Procedure:

- ☐ Open Jupyter Notebook.
- ☐ Import the required libraries (pandas and plotly.express).
- ☐ Load the dataset using `pd.read_csv()`.
- ☐ Display the first five rows using `head()`.
- ☐ Create an interactive scatter plot using `px.scatter()`.
- ☐ Add tooltips using the `hover_data` parameter.
- ☐ Create an interactive line chart using `px.line()`.
- ☐ Add filters using the `color` parameter to categorize data.
- ☐ Display the charts using `fig.show()`.
- ☐ Analyze the interactive visualizations and draw conclusions.

Program:

#import libraries and load dataset

```
import pandas as pd
import plotly.express as px
df = pd.read_csv("Teen_Mental_Health_Dataset.csv")
print(df.head)
```

```
   age  gender  daily_social_media_hours  platform_usage  sleep_hours \
0   14   male                7.9      Instagram        7.4
1   19  female                1.9      TikTok          8.0
2   17  female                1.3      Instagram        7.6
3   15   male                7.4      TikTok          6.9
4   15  female                4.7      Both            4.9

   screen_time_before_sleep  academic_performance  physical_activity \
0                2.9                3.01                1.5
1                2.9                3.22                0.8
2                0.5                3.92                0.0
3                1.6                3.48                0.8
4                3.0                2.37                1.4

   social_interaction_level  stress_level  anxiety_level  addiction_level \
0                low                2                2                1
1                high                8                1               10
2                high                2                4                2
3                medium              1                7                9
4                medium              3                5                2

   depression_label
0                0
1                0
2                0
3                0
4                0
```

#interactive scatter plot

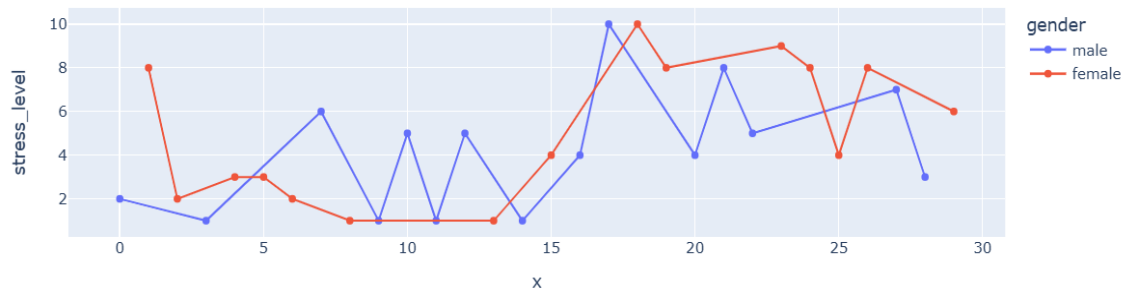
```
fig1 = px.scatter  
( df,  
  x="daily_social_media_hours",  
  y="stress_level", color="gender", size="academic_performance",  
  hover_data=["age", "sleep_hours", "anxiety_level"],  
  title="Social Media Hours vs Stress Level" )
```



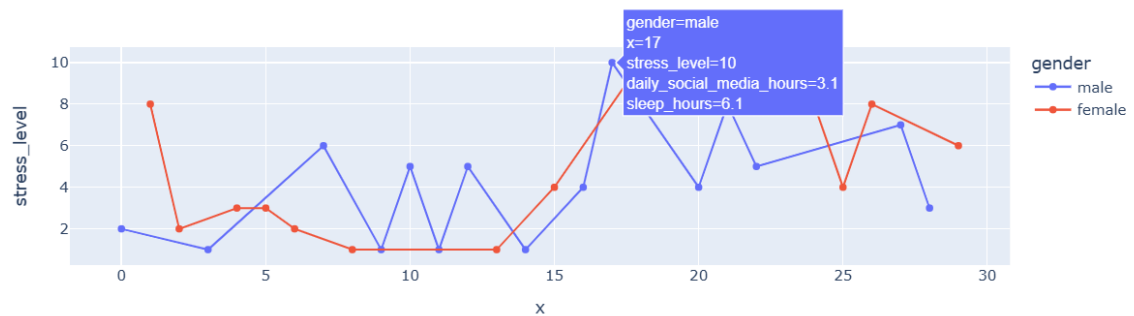
#interactive line chart

```
fig2 = px.line(  
  df.head(30),  
  x=df.head(30).index,  
  y="stress_level",  
  color="gender",  
  markers=True,  
  hover_data=["daily_social_media_hours", "sleep_hours"],  
  title="Stress Level of First 30 Students"  
)  
fig2.show()
```

Stress Level of First 30 Students



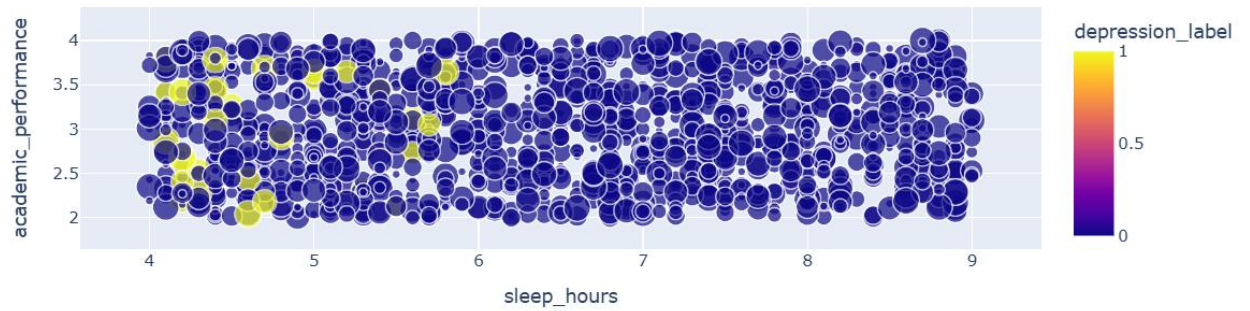
Stress Level of First 30 Students



#interactive dashboard (scatter plot with filter)

```
fig3 = px.scatter(
    df,
    x="sleep_hours",
    y="academic_performance",
    color="depression_label",
    size="stress_level",
    hover_data=[
        "age",
        "gender",
        "daily_social_media_hours",
        "anxiety_level"
    ],
    title="Academic Performance vs Sleep Hours"
)
fig3.show()
```

Academic Performance vs Sleep Hours



Academic Performance vs Sleep Hours

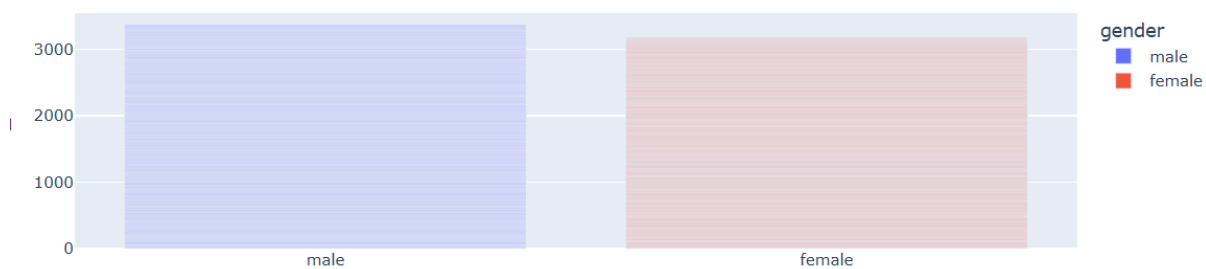


Interactive Bar Chart

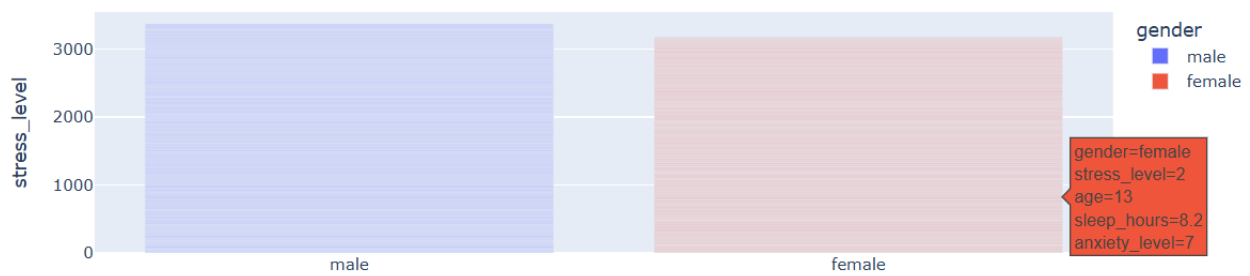
```
fig4 = px.bar(
    df,
    x="gender",
    y="stress_level",
    color="gender",
    hover_data=["age", "sleep_hours", "anxiety_level"],
    title="Average Stress Level by Gender"
)
```

```
fig4.show()
```

Average Stress Level by Gender



Average Stress Level by Gender

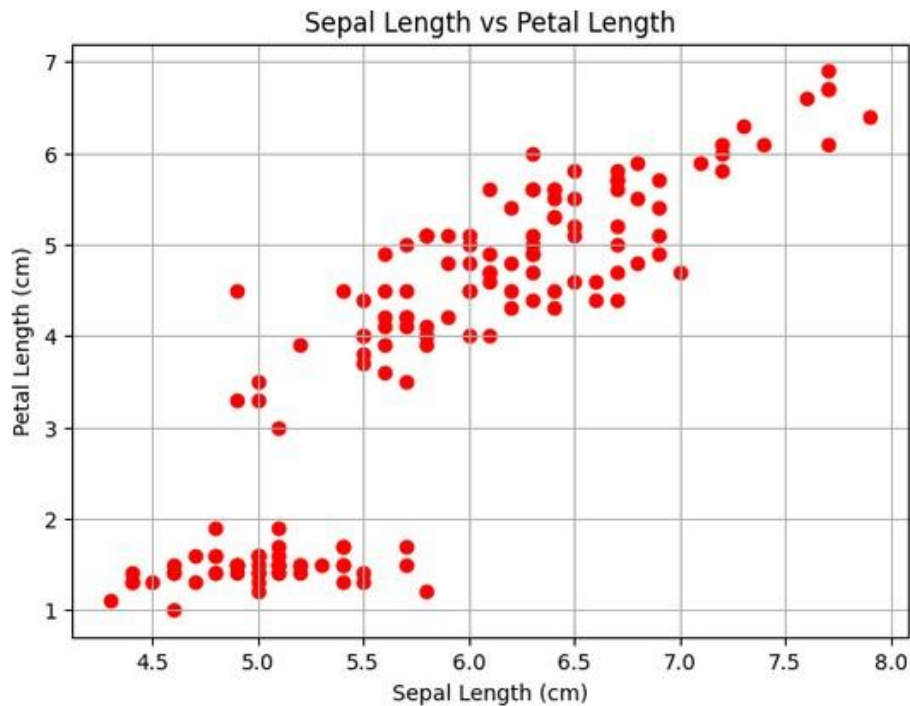


Criterion	Marks Obtained
Record Writing & Presentation	
Problem Understanding & Aim	
Algorithm & Procedure	
Implementation & Visualization	
Output Interpretation & Viva	
Total	

Result

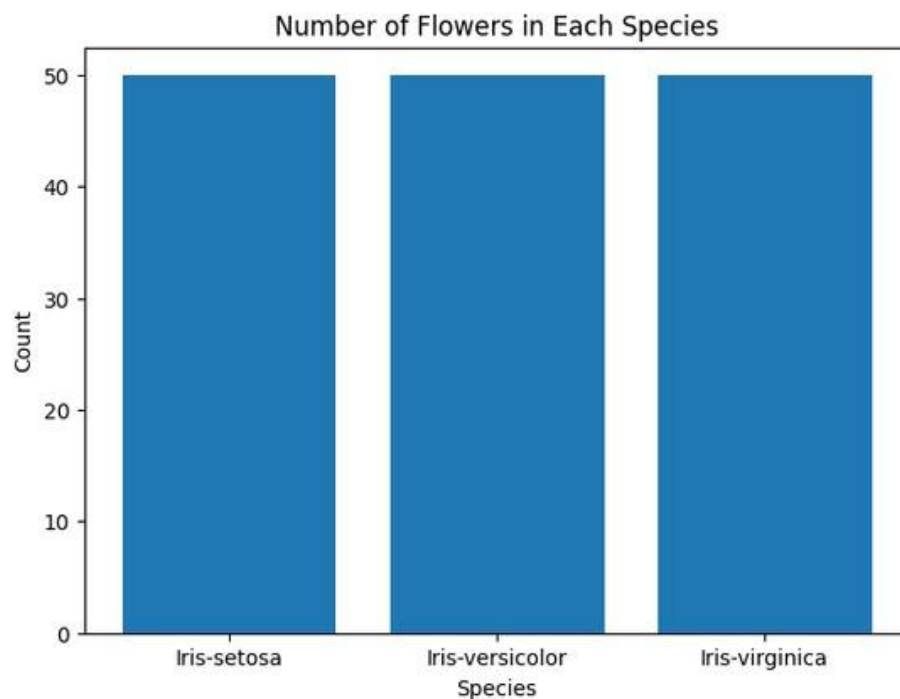
The interactive scatter plot , line chart, and dashboard were Successfully created using Plotly. Tooltips displayed detailed information When hovering over data points, and color-based filtering made it easier to compare different categories in the dataset.

```
#scatter plot plt.figure(figsize=(7,5))  
plt.scatter(df["SepalLengthCm"],  
df["PetalLengthCm"],  
color="red") plt.title("Sepal Length vs  
Petal Length") plt.xlabel("Sepal Length  
(cm)") plt.ylabel("Petal Length (cm)")  
plt.grid(True) plt.show()
```



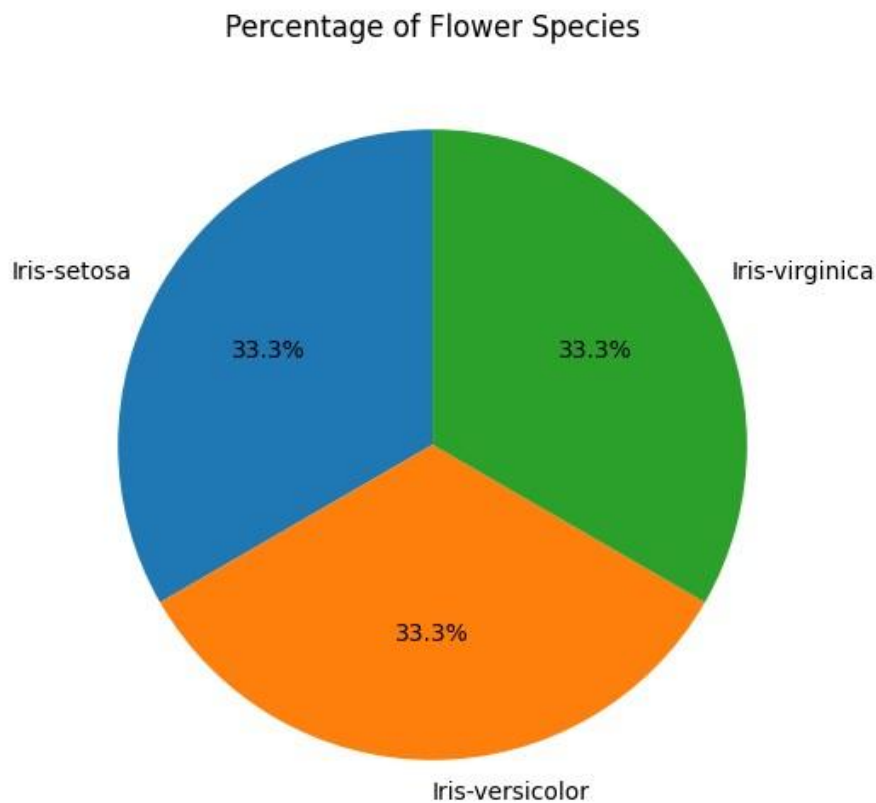
#bar plot

```
species_count = df["Species"].value_counts()
plt.figure(figsize=(7,5))
plt.bar(species_count.index,
species_count.values) plt.title("Number of
Flowers in Each Species")
plt.xlabel("Species") plt.ylabel("Count")
plt.show()
```



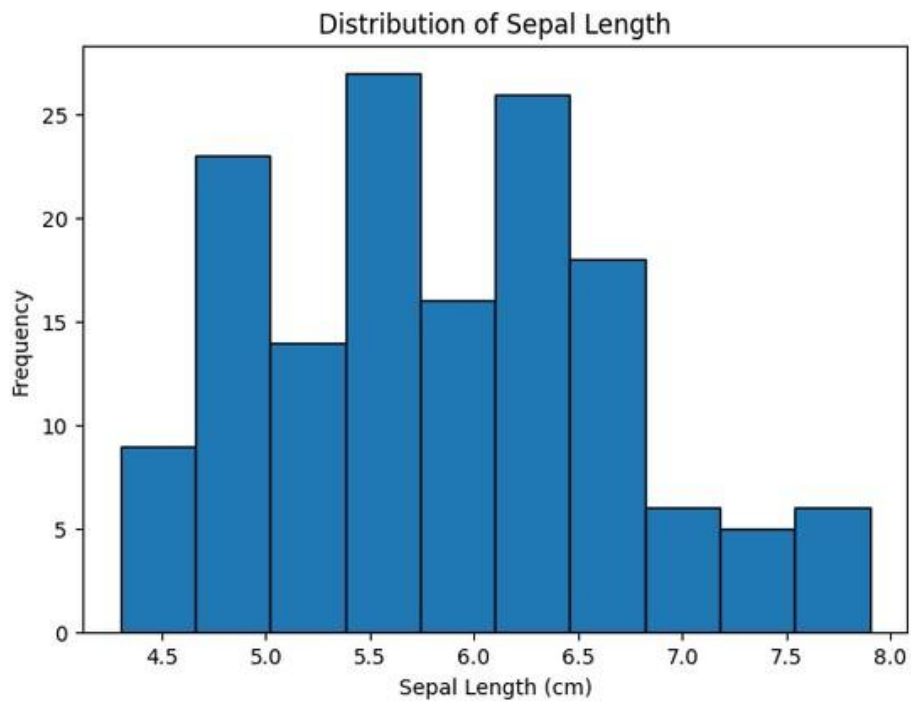
#pie chart

```
species_count =  
df["Species"].value_counts()  
plt.figure(figsize=(6,6))  
plt.pie(species_count.values,  
labels=species_count.index,  
autopct="%1.1f%%", startangle=90)  
plt.title("Percentage of Flower Species")  
plt.show()
```



#histogram

```
plt.figure(figsize=(7,5))  
plt.hist(df["SepalLengthCm"],  
bins=10, edgecolor="black")  
plt.title("Distribution of Sepal  
Length") plt.xlabel("Sepal Length  
(cm)") plt.ylabel("Frequency")  
plt.show()
```



Result:

Simple line, bar, and scatter plots were successfully created using Matplotlib. The charts were properly labeled with titles, axis labels, legends, and grids, demonstrating the basic features of data visualization

EXP NO: 02
DATE:

STATISTICAL PLOTS USING SEABORN

Aim:

To visualize data distributions and relationships using Seaborn statistical plots such as histograms, boxplots, and scatter plots with regression lines, and identify outliers

Procedure:

- Open Google Colab.
- Import the required libraries (seaborn, matplotlib.pyplot, and pandas).
- Load the Iris dataset (or another sample dataset).
- Create a histogram to visualize the distribution of a numerical feature.
- Create a boxplot to identify outliers in the selected feature.
- Plot a scatterplot with a regression line to study the relationship between two variables.
- Add appropriate titles and axis labels.
- Interpret the distribution, trends, and outliers from each graph

Program:

```
# Import Libraries import
pandas as pd import seaborn
as sns import
matplotlib.pyplot as plt
# Read Dataset df =
pd.read_csv("/content/sample_data/Iris.csv")
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

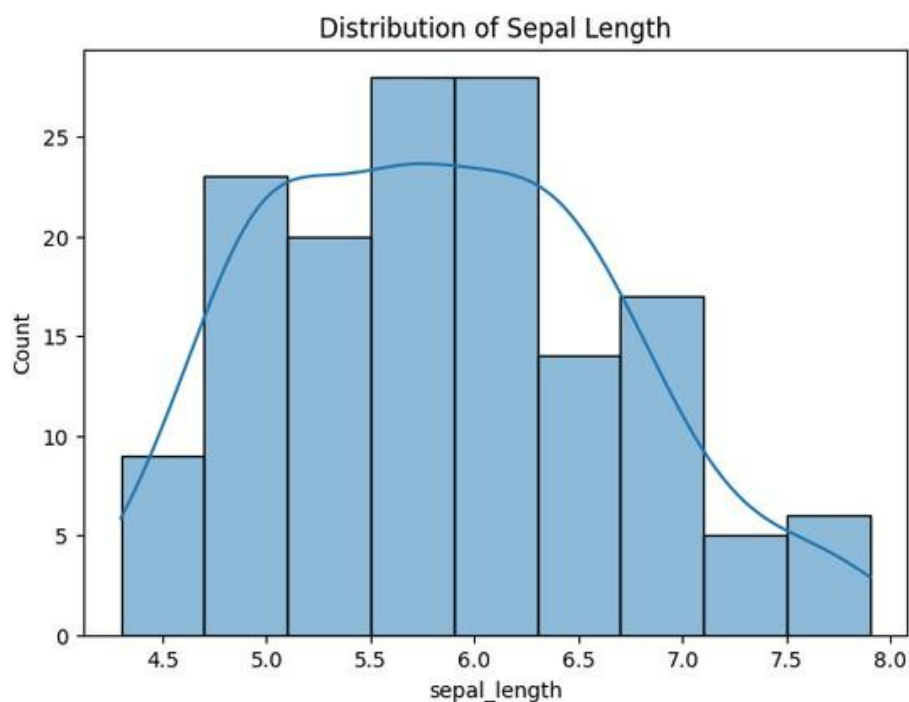
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                    150 non-null   int64
1   SepalLengthCm         150 non-null   float64
2   SepalWidthCm          150 non-null   float64
3   PetalLengthCm         150 non-null   float64
4   PetalWidthCm          150 non-null   float64
5   Species               150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

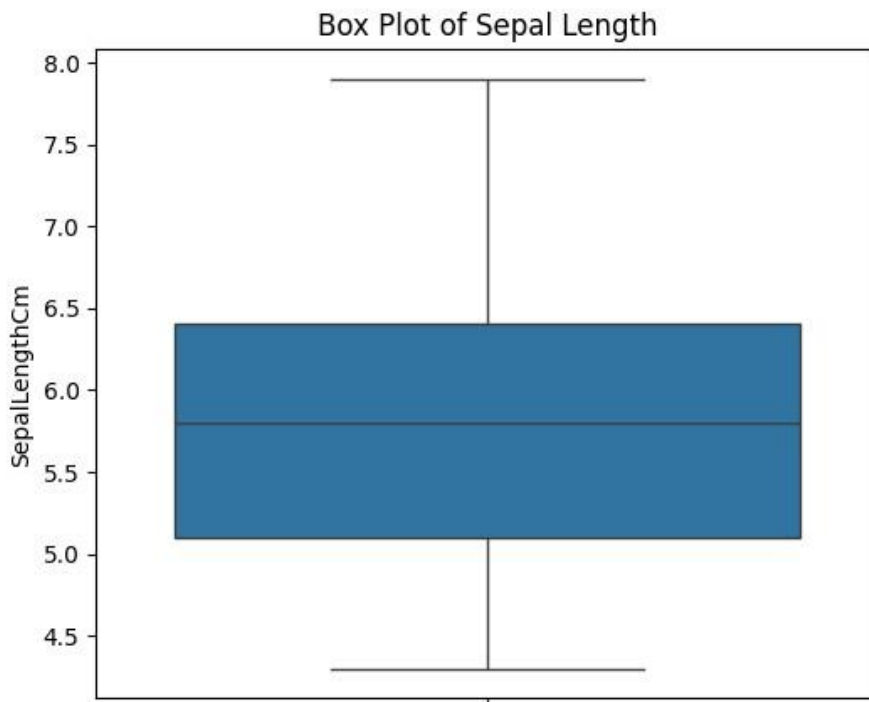
#Histogram with Density Estimation (KDE)

```
plt.figure(figsize=(7,5))
sns.histplot(df["SepalLengthCm"], kde=True)
plt.title("Distribution of Sepal Length")
plt.xlabel("sepal_length") plt.ylabel("Count")
plt.show()
```

Box Plot

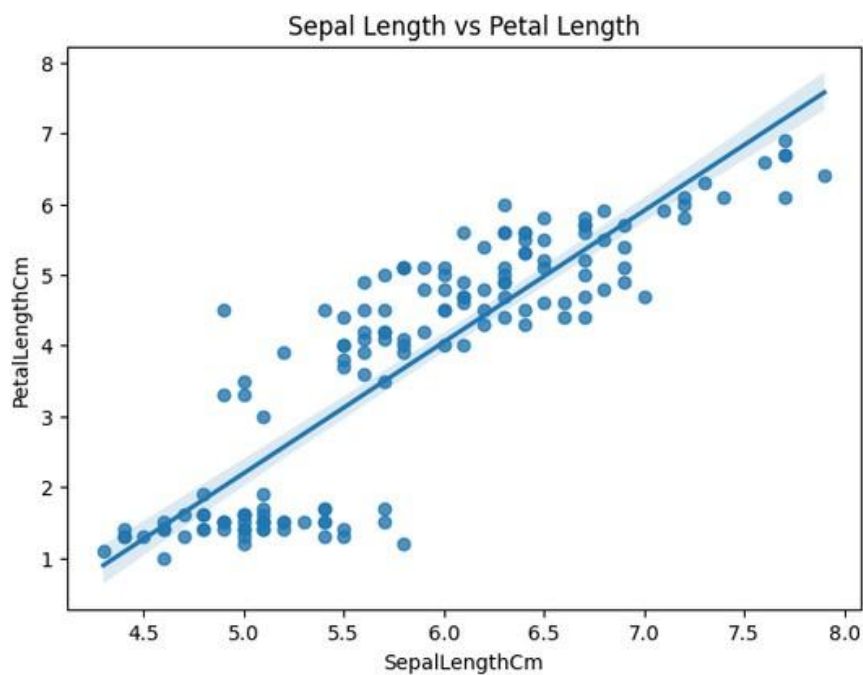


```
plt.figure(figsize=(6,5)) sns.boxplot( y=df["SepalLengthCm"]
)
plt.title("Box Plot of Sepal Length")
plt.show()
```



Scatter Plot with Regression Line

```
plt.figure(figsize=(7,5))  
sns.regplot( data=df,  
x="SepalLengthCm",  
y="PetalLengthCm"  
) plt.title("Sepal Length vs Petal  
Length") plt.show()
```



```
print(df["SepalLengthCm"].sort_values().tail())
```

```
122    7.7
117    7.7
118    7.7
135    7.7
131    7.9
Name: SepalLengthCm, dtype: float64
```

Result:

Histograms, boxplots, and scatter plots with regression lines were successfully generated using Seaborn. The plots clearly showed the distribution of data, detected outliers, and illustrated relationships between variables

EXP NO: 03 DATE:	DATA CLEANING IN PYTHON
-----------------------------------	--------------------------------

Aim:

To perform data cleaning using the Pandas library by identifying and handling missing values and anomalous data, and to visualize the cleaned dataset

Procedure:

- Open Google Colab.
- Import the required libraries (pandas, numpy, matplotlib, and seaborn).
- Load the Titanic dataset into a Pandas DataFrame.
- Examine the dataset using functions like head(), info(), and describe().
- Identify missing values using isnull().sum().
- Handle missing values by:
 - Filling numerical values with the median or mean.
 - Filling categorical values with the mode.
- Detect and remove or treat anomalous (outlier) data if required.
- Plot the distribution of a feature before and after cleaning.
- Compare the results to understand the impact of data cleaning.

Program:**SECTION A: HANDLING MISSING VALUES**

```
# Import Libraries import
pandas as pd import
matplotlib.pyplot as plt
# Read Dataset
df =
pd.read_csv("/content/sample_data/TitanicDataset.csv") df.head() df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   PassengerId     891 non-null    int64
1   Survived        891 non-null    int64
2   Pclass          891 non-null    int64
3   Name            891 non-null    object
4   Sex             891 non-null    object
5   Age            714 non-null    float64
6   SibSp           891 non-null    int64
7   Parch           891 non-null    int64
8   Ticket          891 non-null    object
9   Fare            891 non-null    float64
10  Cabin           204 non-null    object
11  Embarked        889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

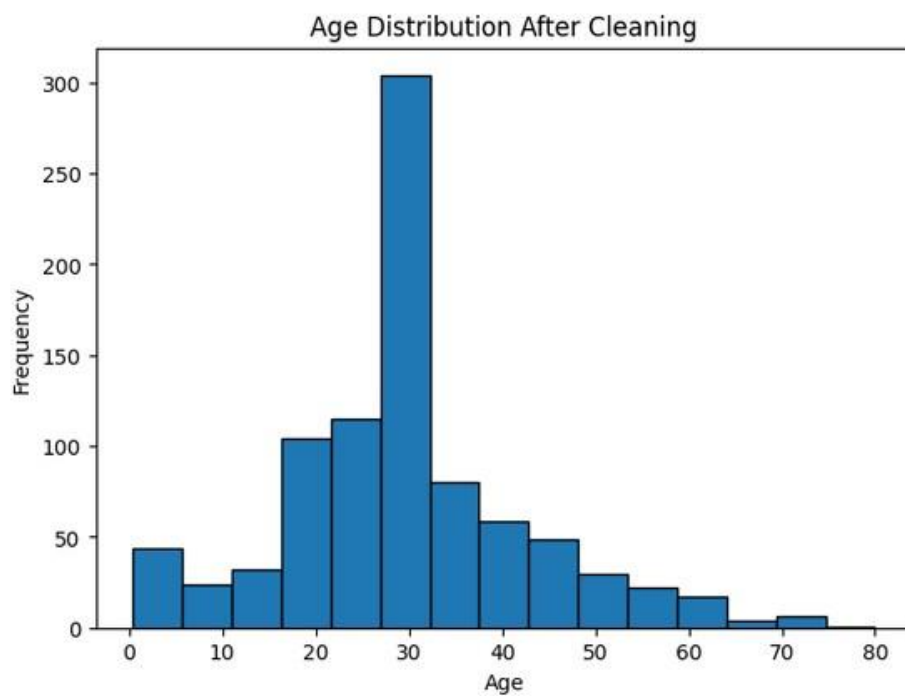
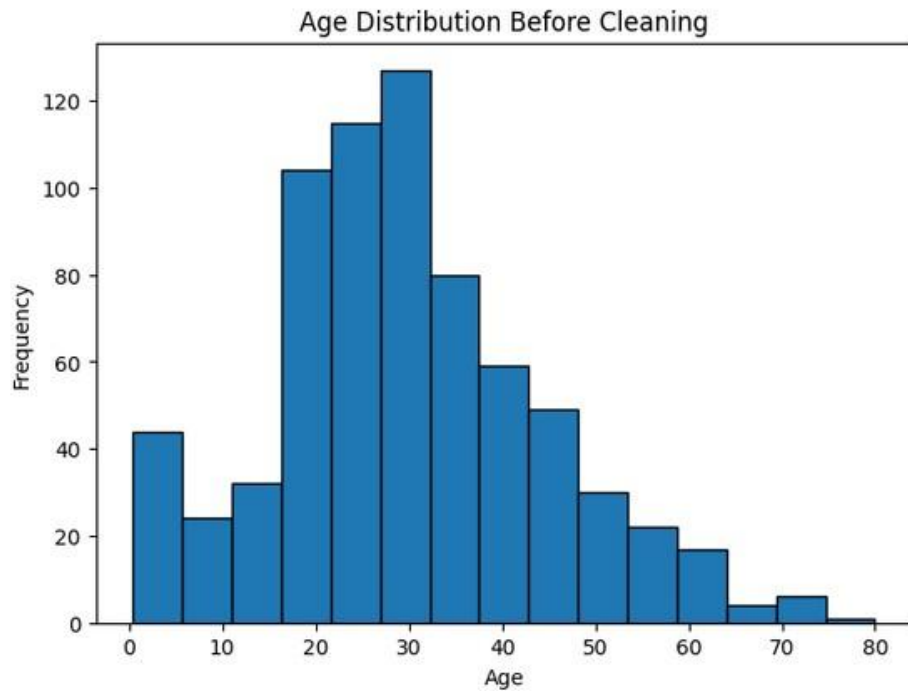
```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

#Visualize Age Distribution (Before and After Cleaning) **#Age Distribution Before Cleaning: # Histogram Before Cleaning** plt.figure(figsize=(7,5))
plt.hist(df["Age"], bins=15, edgecolor="black")
plt.title("Age Distribution Before Cleaning")
plt.xlabel("Age") plt.ylabel("Frequency") plt.show()

Histogram After Cleaning
(Imputing missing values with the median age)
df["Age"].fillna(df["Age"].median(),
inplace=True) plt.figure(figsize=(7,5))
plt.hist(df["Age"], bins=15,
edgecolor="black") plt.title("Age Distribution
After Cleaning") plt.xlabel("Age")
plt.ylabel("Frequency") plt.show()



SECTION B: HANDLING ANOMALOUS (OUTLIER) VALUES

Read Dataset

```
housing =  
pd.read_csv("/content/sample_data/california_housing.csv")  
housing.head() housing.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   MedInc          20640 non-null  float64
1   HouseAge        20640 non-null  float64
2   AveRooms        20640 non-null  float64
3   AveBedrms       20640 non-null  float64
4   Population      20640 non-null  float64
5   AveOccup        20640 non-null  float64
6   Latitude        20640 non-null  float64
7   Longitude       20640 non-null  float64
dtypes: float64(8)
memory usage: 1.3 MB

```

Scatter Plot Before Cleaning

```

plt.figure(figsize=(8,5))
plt.scatter(housing.index,
housing["Population"])
plt.title("Population Before
Cleaning") plt.xlabel("District
Number") plt.ylabel("Population")
plt.grid(True) plt.show()

```

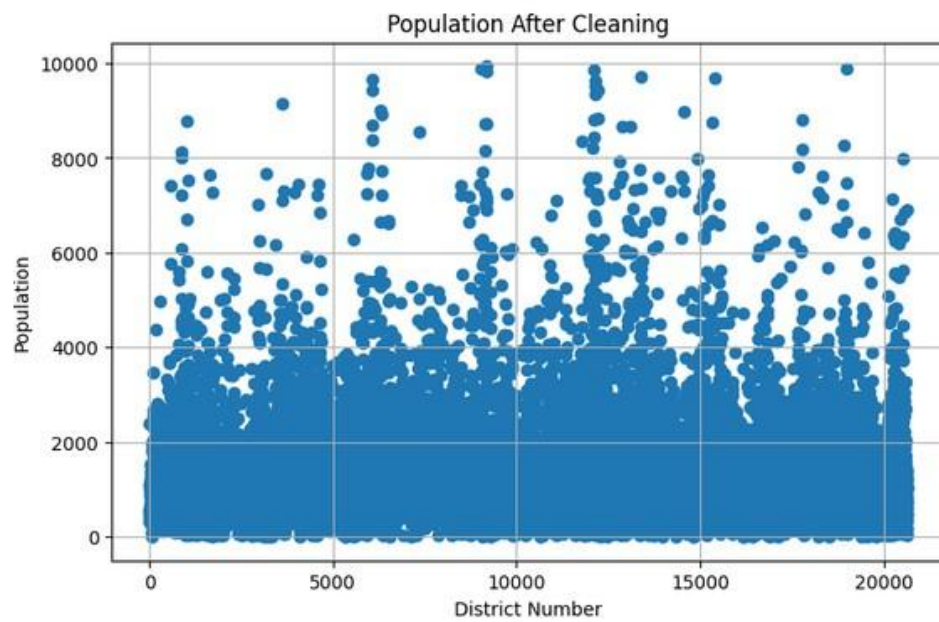
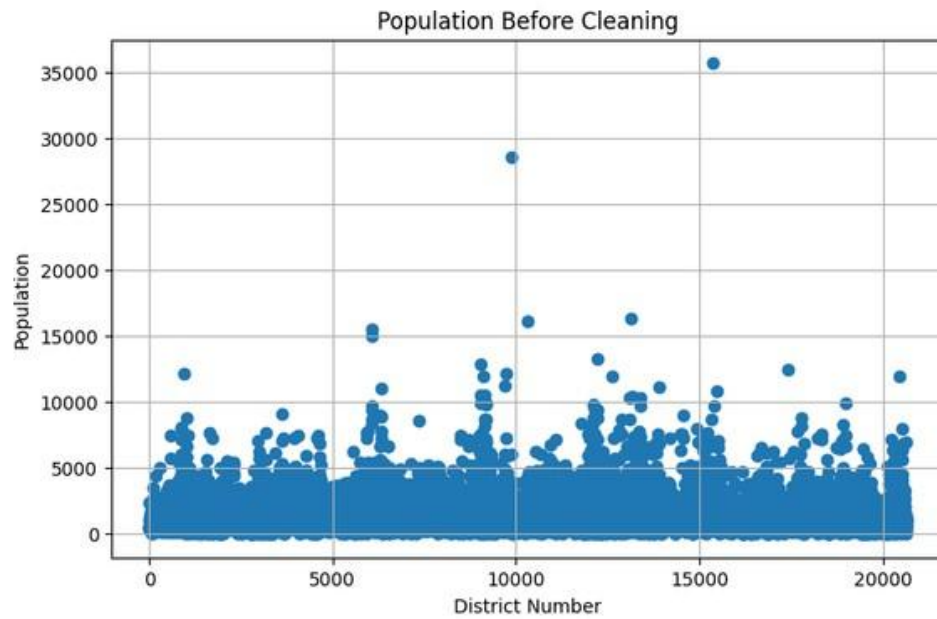
For this experiment, remove districts having population greater than 10,000.
housing_clean = housing[housing["Population"] <= 10000]

Scatter Plot After Cleaning

```

plt.figure(figsize=(8,5))
plt.scatter(housing_clean.index,
housing_clean["Population"])
plt.title("Population After Cleaning")
plt.xlabel("District Number")
plt.ylabel("Population") plt.grid(True) plt.show()
print("Rows Before Cleaning", len(housing))
print("Rows After Cleaning",
len(housing_clean))

```



Rows Before Cleaning 20640

Rows After Cleaning 20617

Result:

Missing values and anomalous data were successfully identified and handled using Pandas. The cleaned dataset showed improved consistency, and the replotted visualizations clearly reflected the impact of the data-cleaning process.